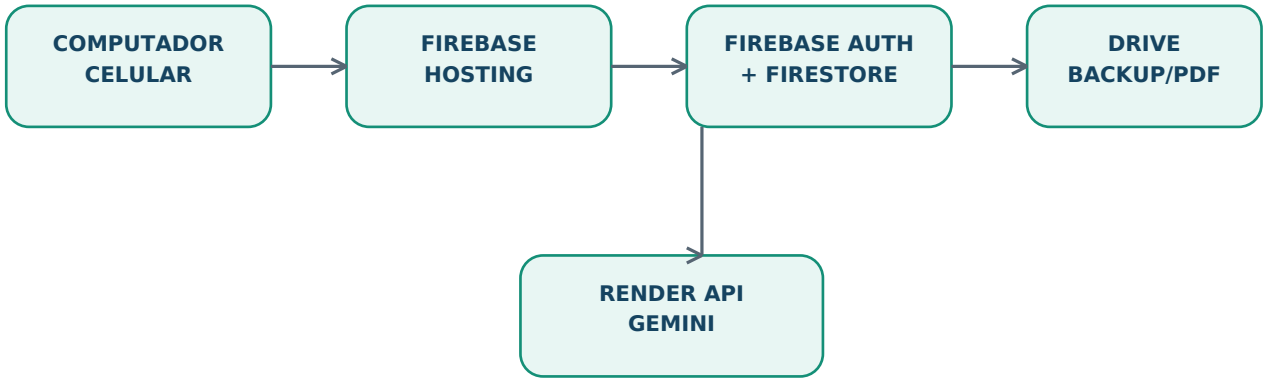
ADAPTAEDU ESCOLAR

Manual técnico para publicação, acesso em computadores e celulares, banco compartilhado e cópias no Google Drive



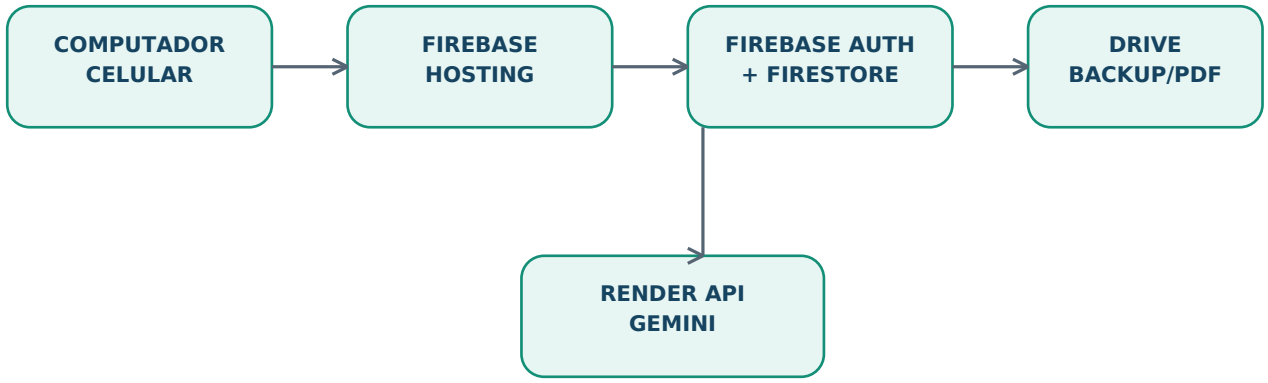
Arquitetura recomendada: dados no Firestore; Drive apenas para arquivos e cópias de segurança.

Dimensionamento: aproximadamente 60 usuários e até 120 estudantes. Versão preparada em 22/10/2026.

Observação importante: o arquivo ZIP não estava disponível no ambiente no momento da elaboração. Portanto, este manual apresenta a arquitetura recomendada e um prompt de auditoria que obriga o Google AI Studio a inspecionar os arquivos reais antes de alterar o projeto.

1. Decisão de arquitetura

Para este porte, use Firebase Hosting no site, Firebase Authentication no login, Cloud Firestore nos dados estruturados e uma API Express no Render somente para operações secretas, principalmente Gemini e integração opcional com o Drive. Não grave o banco principal em arquivos do Drive.



Arquitetura recomendada: dados no Firestore; Drive apenas para arquivos e cópias de segurança.

Por que: o Firestore oferece sincronização e controle de acesso por regras. O Drive é adequado para PDFs, anexos e cópias de segurança, mas não para consultas simultâneas de cadastros, PEIs e diários.

Plano de baixo custo sugerido

Camada	Serviço	Função
Aplicativo web	Firebase Hosting	Publica o React/Vite com HTTPS.
Login	Firebase Authentication	E-mail/senha e identidade de cada usuário.
Banco	Cloud Firestore	Usuários, estudantes, PEIs, diários e atividades.
API secreta	Render Web Service	Gemini, Admin SDK e backup no Drive.
Arquivos	Google Drive institucional	PDFs aprovados e backups exportados.
Código	GitHub privado	Versionamento, rollback e implantação.

A cota gratuita publicada do Firestore inclui 1 GiB armazenado, 50.000 leituras/dia, 20.000 gravações/dia e 20.000 exclusões/dia. Para 60 usuários e 120 estudantes, tende a ser suficiente se as telas forem paginadas e não recarregarem todas as coleções continuamente. Fonte: Firebase Pricing e Firestore Pricing.

2. Prompt completo para o Google AI Studio

Cole este prompt junto com o ZIP descompactado. Ele deve analisar o repositório real antes de modificar qualquer arquivo.

Act as a Principal Software Engineer and Production Cloud Architect specialized in React 19, TypeScript, Vite, Express.js, Firebase Authentication, Cloud Firestore, Firebase Hosting, Google Drive API, Google Gemini API, GitHub and Render.

PROJECT: ADAPTAEDU ESCOLAR. The entire UI, messages, PDFs, database labels and pedagogical content must remain in Brazilian Portuguese (pt-BR).

REAL SCALE: approximately 60 authenticated users and no more than 120 students. Profiles: coordenacao, professor and profissional_apoio.

FIRST, inspect every file in the uploaded repository. Do not generate a new application from scratch. Identify the real package versions, routes, Firebase initialization, Firestore collections, authentication, role model, PEI, support diary, activity generation, PDF export, server, build scripts and deployment configuration. Report errors and affected files before editing.

NON-DESTRUCTIVE RULES:
- Preserve every working feature, layout, route, collection and saved record.
- Create a Git backup branch and rollback instructions before edits.
- Never replace real data with mocks.
- Never invent SEDUC-GO skills, PEI data, diagnoses or diary records.
- Do not expose API keys or Firebase Admin credentials in the browser.
- No TODO, ellipses, partial files or pseudocode.

TARGET ARCHITECTURE:
1. Firebase Hosting serves the React/Vite SPA over HTTPS to computers and mobile phones.
2. Firebase Authentication uses email/password. No public self-registration. Coordination creates or authorizes accounts.
3. Cloud Firestore is the primary shared database. It stores users, students, classes, PEIs, support diaries, activities, audit logs and file metadata.
4. Google Drive is NOT the primary database. Use it only for approved PDFs, attachments and scheduled JSON backup exports in an institutional folder.
5. An Express API on Render handles Gemini calls, Firebase Admin verification and optional Google Drive operations. It must verify Firebase ID tokens and roles for every protected route.
6. Use GitHub as the private source repository and deployment source.

DATA MODEL: preserve the existing model when compatible. Add backward-compatible normalization rather than destructive migrations. Required concepts: users/{uid}, students/{studentId}, peis/{peiId}, supportLogs/{logId}, activities/{activityId}, auditLogs/{logId}, appConfig/main and files/{fileId}. Add schoolId, active, createdAt, updatedAt, createdBy and version where applicable. Never put large PDF base64 strings in Firestore.

SECURITY:
- Firestore Rules version 2 with deny-by-default and role/assignment checks.
- Users cannot change their own role or active status.
- Teachers can access only authorized students/classes and their activities.
- Support professionals can access assigned students and edit only their own diary records.
- Coordination administers users, assignments, students and PEIs.
- Validate allowed fields, ownership and immutable fields.
- Test rules with Firebase Emulator Suite.
- The Express API must independently enforce authorization because Firebase Admin bypasses client Firestore Rules.
- Add App Check where compatible, rate limiting, Helmet, CORS allowlist, body limits, timeouts, safe logs and correlation IDs.

OFFLINE AND MULTI-DEVICE:
- Enable Firestore persistence only with supported-browser error handling.
- Show online/offline/synchronizing states.
- Use serverTimestamp for authoritative timestamps.
- Resolve concurrent edits with updatedAt/version checks or Firestore transactions.
- Avoid device-local storage as the authoritative database.
- Make the interface responsive for 360px mobile screens and desktop.

PERFORMANCE FOR 60 USERS / 120 STUDENTS:
- Paginate lists with query cursors and limits of 20-30 documents.
- Do not attach realtime listeners to entire collections.
- Unsubscribe listeners on component unmount.
- Add the required composite indexes only.

- Cache stable matrix/configuration data.
- Compress images and never store large binary files in Firestore.
- Add loading, empty and retry states.

GOOGLE DRIVE:
- Create a controlled backend-only integration using OAuth 2.0 or a Workspace shared drive/folder approved by the administrator.
- Store Drive folder IDs and file IDs, never public unrestricted links.
- Save generated/approved PDFs under an organized path such as Ano/Estudante/Atividades and backup exports under Backups/YYYY/MM/DD.
- Grant access through Drive permissions only when required. Do not make student files public.
- If secure automatic Drive integration cannot be completed without billing, domain administration or OAuth consent, keep manual/admin-only export and state the exact limitation.
- Implement restore validation and never overwrite Firestore automatically from a backup.

DEPLOYMENT:
- Firebase Hosting public directory must be dist and SPA routes rewrite to /index.html.
- Do not rewrite /api failures to index.html.
- Render must use the real build/start commands, process.env.PORT and 0.0.0.0.
- Render free services sleep after inactivity and have an ephemeral local filesystem; therefore no persistent file or database may be stored locally.
- Use environment variables for

GEMINI_API_KEY, Firebase Admin credentials, allowed origins and Drive settings.
- Provide firebase.json, .firebaserc, firestore.rules, firestore.indexes.json, render.yaml and .env.example when required.

BACKUP:
- Add an admin-only JSON export that includes schemaVersion, exportedAt, counts and a SHA-256 manifest.
- Copy approved exports to Drive, but redact or encrypt sensitive content when institutionally required.
- Provide a restore dry-run that validates schema, IDs and references before writing.
- Keep audit logs for export and restore operations.

MANDATORY TESTS:
- npm install, typecheck, lint, tests and production build.
- Firebase Emulator tests for every role and denied operation.
- Login and password recovery.
- CRUD and cross-device synchronization.
- 120-student seed only in emulator, never production.
- Mobile widths 360, 390 and 430px; desktop 1366px.
- PDF including SVG/images and bastao typography.
- Gemini authorization and malformed response handling.
- Drive upload, permission, metadata and restore dry-run.
- Render cold start and Firebase Hosting SPA refresh.
- Verify secrets are absent from dist and Git history.

OUTPUT ORDER:
A. Repository audit and risk list.
B. Backup and rollback plan.
C. Architecture found versus target architecture.
D. Complete corrected files, without truncation.
E. Firebase Console click-by-click setup.
F. GitHub and Render click-by-click deployment.
G. Google Drive integration setup.
H. Firestore Rules and Emulator test results.
I. Production validation checklist.
J. Rollback instructions.

Use PASS, FAIL or NOT EXECUTED. Never claim a test passed unless it was actually executed. If official matrix files or credentials are missing, state exactly what is missing and continue all safe work without fabricating data.

3. Preparação antes de publicar

• Crie uma conta institucional administradora para o projeto, evitando que toda a infraestrutura dependa da conta pessoal de um único servidor.

• Descompacte o ZIP em uma pasta simples, por exemplo C:\AdaptaEdu, sem acentos no caminho.

• Instale Node.js LTS, Git e Visual Studio Code.

• Faça uma cópia integral da pasta antes de executar correções automáticas.

• Não envie arquivos .env, chaves JSON ou credenciais ao GitHub.

Comandos iniciais no terminal do VS Code

```
git init
git checkout -b backup-antes-publicacao
git add .
git commit -m "backup: versão antes da publicação"
npm install
npm run typecheck
npm run build
```

Se algum comando falhar, não publique. Copie a mensagem completa e peça ao Google AI Studio para corrigir a causa sem remover funcionalidades.

4. Criar o projeto no Firebase

Clique a clique

• Acesse console.firebase.google.com e clique em “Adicionar projeto”.

• Nome sugerido: adaptaedu-escolar-olynto. Anote o ID real do projeto.

• Abra “Configurações do projeto” > “Seus aplicativos” > ícone Web .

• Registre o app como AdaptaEdu Web. Não marque Hosting ainda se preferir configurar pela CLI.

• Copie firebaseConfig para o arquivo de configuração do frontend. Esses identificadores podem estar no navegador; as regras protegem os dados.

• Abra “Authentication” > “Primeiros passos” > “E-mail/senha” > ative somente E-mail/senha.

• Abra “Firestore Database” > “Criar banco de dados” > modo Produção. Escolha uma região próxima e compatível com o projeto; a região não pode ser alterada depois.

• Não use regras allow read, write: if true. Publique regras por perfil antes de cadastrar dados reais.

5. Modelo mínimo de dados

Coleção	Documento/chave	Observação
users	UID do Authentication	nome, email, role, active, schoolId
students	ID automático	nome, turmaId, active; evite dados desnecessários
peis	ID automático	studentId, versão, status, campos pedagógicos
supportLogs	ID automático	studentId, authorUid, timestamp, observações
activities	ID automático	studentId, teacherUid, habilidade, conteúdo
auditLogs	ID automático	ação, actorUid, timestamp, correlationId
files	ID automático	driveFileId, ownerType, ownerId, mimeType

Nunca grave PDF ou imagem grande em Base64 no Firestore. Salve apenas metadados e o identificador do arquivo.

6. Publicar o frontend no Firebase Hosting

No terminal, dentro da pasta do projeto:

```
npm install -g firebase-tools
firebase login
firebase use --add
firebase init hosting
```

• Escolha “Use an existing project”.

• Selecione o projeto AdaptaEdu.

• Public directory: dist.

• Configure as single-page app: Yes.

• Set up automatic builds with GitHub: escolha No na primeira publicação, para reduzir variáveis.

• Se perguntado sobre sobrescrever dist/index.html, escolha No.

Confirme firebase.json

```
{
"hosting": {
"public": "dist",
"ignore": ["firebase.json", "**/.*", "**/node_modules/**", "**/*.map"],
"rewrites": [{ "source": "**", "destination": "/index.html" }]
},
"firestore": {
"rules": "firestore.rules",
"indexes": "firestore.indexes.json"
}
}
```

```
npm run build
firebase deploy --only hosting,firestore:rules,firestore:indexes
```

Ao terminar, copie o endereço terminado em web.app. Abra em janela anônima, outro computador e celular usando 4G/5G.

7. GitHub privado e implantação do backend no Render

• Crie um repositório privado no GitHub.

• Confirme que .gitignore contém .env, .env.*, serviceAccount*.json, node_modules e dist.

• Envie o código com git remote add origin, git branch -M main e git push -u origin main.

• No Render, clique New + > Web Service > conecte o GitHub > selecione o repositório.

• Environment: Node. Branch: main. Build Command: npm install && npm run build. Start Command: npm start.

• Selecione Free apenas para piloto. O serviço gratuito entra em repouso após inatividade e pode demorar cerca de um minuto para acordar; o próprio Render não o recomenda como produção.

• Cadastre variáveis secretas em Environment. Nunca coloque credenciais no código.

• Defina Health Check Path como /api/health.

• Após publicar, copie a URL onrender.com para VITE_API_URL no ambiente de build do frontend e refaça o deploy do Hosting.

Variáveis típicas do Render

```
NODE_ENV=production
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-2.5-flash
FIREBASE_PROJECT_ID=...
FIREBASE_CLIENT_EMAIL=...
FIREBASE_PRIVATE_KEY=...
ALLOWED_ORIGINS=https://SEU-PROJETO.web.app,https://SEU-PROJETO.firebaseapp.com
GOOGLE_DRIVE_FOLDER_ID=...
```

A chave privada costuma conter quebras de linha. O backend deve converter \n para nova linha de modo controlado. Nunca copie o JSON completo para o frontend.

8. Google Drive: arquivos e backup, não banco principal

Opção recomendada para começar

• Crie no Drive institucional a pasta “AdaptaEdu Escolar”.

• Dentro dela, crie “PDFs Aprovados”, “Anexos” e “Backups”.

• Compartilhe a pasta apenas com a equipe autorizada. Não use “qualquer pessoa com o link” para documentos de estudantes.

• Na primeira versão, permita que a Coordenação baixe o backup JSON e o envie manualmente ao Drive. Isso é mais simples e reduz riscos de OAuth.

• Depois, implemente integração automática pelo backend com Google Drive API e consentimento institucional.

Fluxo correto

```
USUÁRIO → APP WEB → FIRESTORE (dados vivos)
                  ■ API SEGURA → DRIVE (PDF/backup)

RESTAURAÇÃO: DRIVE → validação a seco → relatório → autorização da Coordenação → Firestore
```

Nunca restaure automaticamente um arquivo do Drive sobre o banco. Primeiro valide versão do esquema, contagens, IDs, referências e hash.

9. Controle de usuários e permissões

• Desative cadastro público. A Coordenação cria ou autoriza os 60 usuários.

• Documento users/{uid}: role deve ser coordenacao, professor ou profissional_apoio.

• Um usuário não pode alterar o próprio role ou active.

• Use atribuições explícitas de turmas/estudantes. Não permita que todo professor leia todos os PEIs sem necessidade.

• Teste cada regra no Emulator Suite e também com três contas reais de teste.

• Registre no auditLogs alterações de PEI, exclusões, geração de atividade, exportação e restauração.

10. Estimativa de uso e prevenção de estouro

Exemplo conservador: 60 usuários × 100 leituras/dia = 6.000 leituras/dia; 60 × 20 gravações/dia = 1.200 gravações/dia. Isso permanece abaixo das cotas gratuitas publicadas de 50.000 leituras e 20.000 gravações por dia. A estimativa depende do código: uma tela que lê coleções inteiras repetidamente pode multiplicar o consumo.

• Carregue 20 ou 30 registros por página.

• Use query com where, orderBy e limit.

• Não mantenha listeners em coleções completas.

• Feche listeners ao sair da tela.

• Não leia o mesmo PEI em vários componentes simultaneamente.

• Monitore Usage no console do Firestore semanalmente no primeiro mês.

• Mantenha um limite de tamanho para anexos e comprima imagens.

11. Teste final em outros computadores e celulares

☐ 1. O endereço web.app abre sem aviso de certificado.

☐ 2. O login funciona em Chrome, Edge e navegador do celular.

☐ 3. Atualizar uma rota interna não produz erro 404.

☐ 4. Usuário não autenticado não lê Firestore.

☐ 5. Professor não acessa estudante não atribuído.

☐ 6. Profissional de apoio edita apenas seus registros.

☐ 7. Coordenação cria/desativa usuário sem apagar histórico.

☐ 8. Uma alteração salva no computador aparece no celular.

☐ 9. O indicador offline aparece sem internet e sincroniza depois.

☐ 10. Atividade com imagem e letra bastão é gerada.

☐ 11. PDF A4 não apresenta sobreposição nem imagem ausente.

☐ 12. Gemini não expõe chave no DevTools.

☐ 13. Backup JSON é gerado e armazenado no Drive.

☐ 14. Restauração a seco aponta erros sem alterar produção.

☐ 15. Logout encerra a sessão e volta à tela de login.

12. Rotina operacional

Diariamente

• Verificar se login, gravação e geração de atividade funcionam.

• Confirmar falhas no painel/logs sem expor dados pessoais.

Semanalmente

• Exportar backup administrativo e guardar em pasta datada no Drive.

• Verificar cotas de Firestore, Hosting e Render.

• Revisar usuários inativos e acessos indevidos.

Mensalmente

• Testar restauração a seco do backup mais recente.

• Atualizar dependências em uma branch separada e repetir todos os testes.

• Revisar regras, índices, logs e contas vinculadas.

13. Limitações reais do plano gratuito

O Firebase Hosting é adequado para publicar a SPA com HTTPS e CDN. Firestore e Authentication possuem cotas gratuitas amplas para este porte, se o aplicativo for eficiente. Já o Render alerta que instâncias gratuitas são destinadas a testes/hobby, adormecem após 15 minutos sem tráfego e usam sistema de arquivos efêmero. Portanto, não salve arquivos no disco do Render e espere atraso no primeiro acesso à API após repouso.

Se a escola exigir disponibilidade imediata e constante para geração com IA, o primeiro upgrade recomendado é somente o Web Service do Render para o menor plano pago disponível. O frontend e o banco podem permanecer nas cotas gratuitas enquanto o uso estiver abaixo dos limites.

14. Checklist de publicação

☐ ZIP auditado e backup criado

☐ npm install concluído

☐ typecheck sem erros

☐ testes sem erros

☐ build criado

☐ Firebase Authentication ativado

☐ Firestore em produção

☐ regras por perfil publicadas

☐ índices publicados

☐ Hosting publicado

☐ GitHub privado sem segredos

☐ Render publicado e health check OK

☐ origens CORS corretas

☐ URL da API no frontend

☐ Drive com permissões restritas

☐ backup e restore dry-run testados

☐ teste em outro computador

☐ teste em celular com rede móvel

☐ PDF e imagens validados

☐ plano de rollback impresso

15. Fontes oficiais consultadas

Firebase Pricing: https://firebase.google.com/pricing

Firestore pricing and free quota: https://cloud.google.com/firestore/pricing

Firebase Authentication limits: https://firebase.google.com/docs/auth/limits

Firebase Hosting: https://firebase.google.com/docs/hosting/

Firebase Hosting quickstart: https://firebase.google.com/docs/hosting/quickstart

Firestore Security Rules: https://firebase.google.com/docs/firestore/security/get-started

Render free services: https://render.com/docs/free

Render pricing: https://render.com/pricing

Google Drive API overview: https://developers.google.com/workspace/drive/api/guides/about-sdk

Google Drive permissions: https://developers.google.com/workspace/drive/api/guides/manage-sharing

Este manual deve ser atualizado depois da auditoria do ZIP, porque nomes de arquivos, comandos e coleções precisam refletir exatamente o código real.